

哈希表

前言

链表和数组

数组：空间连续 长度固定，基于下标的搜索可以 $O(1)$ ，增删麻烦

链表：增删灵活，查找麻烦

要是可以结合在一起就好了。。。

Hash Table（散列表）

功能：无序数据，快速搜索 $O(1)$

哈希表可以直接使用 `unordered_map` / `unordered_set`

hash table的底层实现

散列：按照数据的某种特性进行分组

建哈希表：

1. 确定散列函数 常用： $p = key \% M$ 其中，M一般取质数，并小于等于N（理论上），实际都是大于N的。（建议了解 `unordered_map` / `unordered_set` 是怎么实现的）
2. 解决哈希冲突的办法（一组来了俩人）
 - 开放地址法：尽量不产生冲突的前提下才会用（不咋用）
 - 线性探测：沿着当前元素的位置向后移动直到有空位置
 - 线性补偿探测：往后跳着找，一般不用，容易死循环
 - 二次探测：按照 $\pm 1 \pm 4 \pm 9 \pm 16 \pm 25$ 顺序探测
 - 拉链法：数组挂着链表，来都来了，用链表形式链下来，对冲突的情况友好

哈希表建立要什么：

1. 节点类
 - value
 - next
2. 哈希表
 - 地址数组（M个 `nullptr`）
 - 完成元素添加：先用哈希函数获得对应组；向该链表中进行节点添加；

```
#include <iostream>
#include <vector>
using namespace std;
```

```

#define M 11

class node
{
public:
    int value;
    node* next;
    node(int num) :value(num), next(nullptr) {}
};

class HashTable
{
public:
    vector<node*>hash;
    HashTable() :hash(M, nullptr) {}
    void CreateHash(vector<int>&nums)
    {
        int len = nums.size();
        if (len == 0) return;
        for (int v : nums)
        {
            //申请节点
            node* tmp = new node(v);
            //位置确定
            int index = v % M;
            //链表结点的通知
            tmp->next = hash[index];
            hash[index] = tmp;
        }
    }
    void Search(int num)
    {
        int index = num % M;
        node* tmp = hash[index];
        //遍历链表
        while (tmp)
        {
            if (tmp->value == num)
            {
                cout << "success" << endl;
                return;
            }
            tmp = tmp->next;
        }
        cout << "failed" << endl;
    }
};

int main()
{
    vector<int>nums = { 12,44,13,88,23,94,11,39,20,16,5 };
}

```

```
HashTable ht;
ht.CreateHash(nums);
ht.Search(12);
ht.Search(44);
ht.Search(100);
return 0;
}
```

哈希表实现的优化

开放地址法的装载因子： $\alpha = \frac{\text{元素个数}}{\text{表长}}$ ，在线性探测里， $\alpha < 0.7 \sim 0.8$

添加的话，只能数组扩容，申请一块更大的空间，将原来的东西拷贝过去，但是要重新散列，取得新位置

✍ 扩容过程中是怎么迁移数据的？

旧表向新表迁移，不是一次全迁移完，而是分批次，这会导致同一时间段内新旧表同时存在，这时候查找就会：先查新后查旧；

✍ 删除操作

删除操作中，不是随便覆盖的

哈希表的元素中不只放元素，还会放状态state，删除的时候将state置为删除态（开放地址法不好用就在这里，迁移和删除很麻烦）

拉链法的装载因子： $\alpha < 1$ 越接近1，冲突可能性越高

✍ C语言下 拉链法扩容机制

不用拷贝，直接把旧链表节点拆下来插到新表合适的位置上，旧表指向新表，新表指向旧表，最后释放旧表，这样old标记的就是新表了

【注】JAVA下 拉链法哈希表扩容机制
java下规定一个单向链表不能超过8个节点，超过了会自动折叠成红黑树的形态

拉链法和线性探测

拉链法的优势：

1. 处理冲突简单
2. 删除容易
3. 链表不限制个数，更适合未知元素个数的处理

线性探测：以上有点全没有

桶排序 Bucket Sort

给 $[0, 1)$ 区间内的相同位数的小数排序用

核心思想：按区间进行分组，组与组之间要求有序，对每组内进行排序

类似于哈希的操作，先放到桶里，桶内链表冒泡排序

内部排序把数据放内存内排序，外部排序借助外面磁盘排序

基数排序 Radix Sort

LSD 低位优先

思路：元素在高位相同的情况下，低位的顺序就是高位的顺序

MAD 高位优先（一般不用）

先按照个位分组**并维持原有相对顺序不变**，再十位分组，以此类推